Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Семантическая сегментация изображений с использованием архитектуры U-Net

Данная лабораторная работа посвящена семантической сегментации снимков дистанционного зондирования земли с помощью модели нейронной сети U-Net.

Цель работы: получить навыки сегментации снимков дистанционного зондирования земли с помощью модели нейронной сети U-Net.

Задачи:

1. Ознакомиться с работой сверточных нейронных сетей в библиотеке Keras;
2. Научиться решать задачу сегментации изображений на основе нейронных сетей.
3. Научиться применять модель сверточной нейронной сети U-Net.

Модель сверточной нейронной сети U-Net

U-Net – это сверточный автоэнкодер, который был создан в 2015 году для сегментации биомедицинских изображений в отделении Computer Science Фрайбургского университета. Архитектура сети представляет собой полносвязную свёрточную сеть, модифицированную так, чтобы она могла работать с меньшим количеством примеров (обучающих образов) и обеспечивала более точную сегментацию. Данная нейронная сеть содержит сжимающий путь (кодировщик) и расширяющий путь (декодировщик). Более подробное описание архитектуры U-Net с её программной реализацией представлено в разделе 8.

Обучающая выборка

В качестве обучающей и тестовой выборки используется набор изображений **Humans in the Loop**, который разрабатывался для совместного проекта с Космическим центром Мохаммеда Бин Рашида в Дубае, ОАЭ и находится в открытом доступе. Набор данных состоит из аэрофотоснимков Дубая, полученных со спутников MBRSC и аннотированных с помощью семантической сегментации.

Пример входных данных можно увидеть слева на рисунке 1, справа пример семантической сегментации.

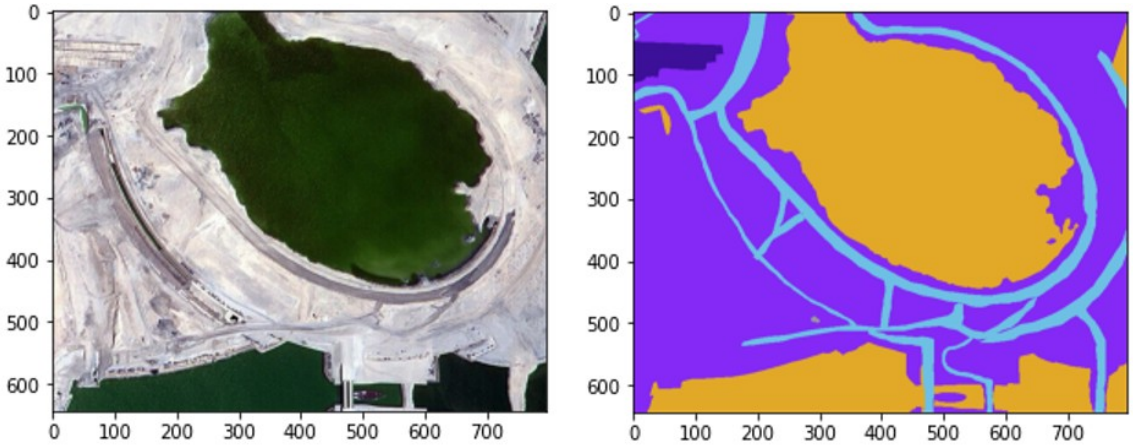


Рисунок 1. Пример изображений для обучения нейронной сети

Ход работы

1. Загрузка данных

Оригинальную базу изображений HITL можно скачать на официальном сайте: https://humansintheloop.org/uae-dataset-download/?submissionGuid=4d1367f9-97ed-4c5b-b1ff-b6a60749290a

Что бы не проходить регистрацию вы можете скачать ее по ссылке: https://drive.google.com/drive/folders/1ZFOn56fko9Na1WJ7rYCbxgKMvs8NQIBE?usp=sharing

Далее вам необходимо загрузить выборку данных на свой Google диск.

Чтобы подключить Вашу программу к Google диску, вы должны вставить и запустить следующий блок кода:

```
from google.colab import drive
drive.mount('/content/drive')
```

Запустите код. После запуска появится ссылка доступа, перейдите по ней для авторизации. Затем, приложение **«Google Drive File Stream»** запросит разрешение на доступ к вашему аккаунту Google, нажмите «разрешить».

Для начала работы выполним загрузку необходимых библиотек.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageDraw
from skimage import io
import random
import os
```

NumPy — это библиотека языка Python, добавляющая поддержку больших многомерных массивов и матриц.

Matplotlib — пользовательская библиотека для построения графиков на языке python.

skiimage — это Pythonпакет с открытым кодом, который работает с массивами NumPy. Он реализует алгоритмы и утилиты для использования в исследовательских, образовательных и промышленных приложениях.

Метод **os** в python используется для получения списка всех файлов и каталогов в указанном каталоге.

```
def download_data(path):
  data = []
  for path_image in sorted(os.listdir(path=path)):
      image = Image.open(path + path_image)
      data.append(np.array(image))
  return data

X_train = download_data(r"./drive/My Drive/HITL/train/images/")
Y_train = download_data(r"./drive/My Drive/HITL/train/masks/")
X_test = download_data(r"./drive/My Drive/HITL/train/images/")
Y_test = download_data(r"./drive/My Drive/HITL/train/masks/")
```

Данный блок позволяет загрузить изображения выборки данных из Google диска в переменные X_train, Y_train, X_test, Y_test.

Где:

X_train – Исходные изображения из обучающей выборки данных.

Y_train – Сегментированные исходные изображения из обучающей выборки.

X_test – Исходные изображения из тестовой выборки данных.

Y_test – Сегментированные исходные изображения из тестовой выборки.

2. Вывод загруженных изображений на экран

Для того чтобы проверить правильность загрузки наших данных выведем несколько загруженных изображений и их соответствующие маски (рисунок 2).

```
num_images = 6
random_indices = random.sample(range(len(X_train)), num_images)

plt.figure(figsize=(15, 5))

for i, idx in enumerate(random_indices):
    plt.subplot(2, num_images, i + 1)
    plt.imshow(X_train[idx])
    plt.title(f"Изображение {idx}")
    plt.axis('off')

    plt.subplot(2, num_images, num_images + i + 1)
    plt.imshow(Y_train[idx])
    plt.title(f"Маска {idx}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

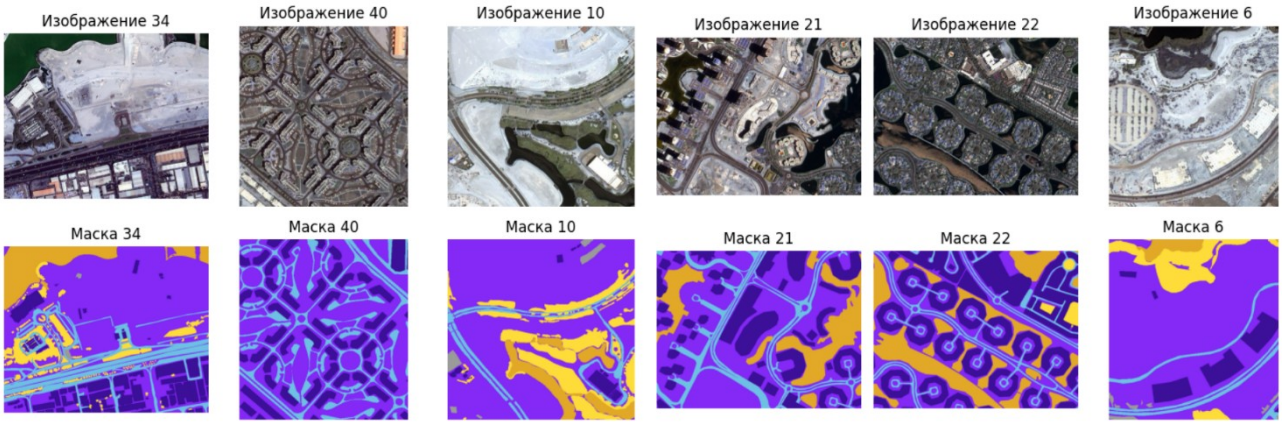


Рисунок 2. Вывод изображений из обучающей выборки

3. Предобработка данных

Для определения цветов, в которые будет сегментироваться изображение вставьте следующий блок кода:

```
palette = {0 : (60, 16, 152), # Building
           1 : (132, 41, 246), # Land
```

```
            2 : (110, 193, 228), # Road
            3 : (254, 221, 58), # Vegetation
            4 : (226, 169, 41), # Water
            5 : (155, 155, 155)} # Unlabeled

invert_palette = {v: k for k, v in palette.items()}

def convert_to_color(arr_2d, palette=palette):
    arr_3d = np.zeros((arr_2d.shape[0], arr_2d.shape[1], 3), dtype=np.uint8)

    for c, i in palette.items():
        m = arr_2d == c
        arr_3d[m] = i
    return arr_3d
```

Создается словарь **palette**, где ключи – это метки классов, а значения – соответствующие этим меткам RGB-цвета.

Создается новый словарь **invert_palette**, где ключи и значения из **palette** меняются местами. Теперь ключами являются RGB-цвета, а значениями – соответствующие метки классов.

Создадим функцию **convert_to_color**, которая принимает два аргумента: **arr_2d** – двумерный массив, где каждый элемент – это метка класса (число от 0 до 5); **palette** – словарь, который связывает метки классов с RGB-цветами.

Функция **convert_to_color** преобразует двумерный массив меток классов (arr_2d) в трехмерный массив RGB-изображения (arr_3d), используя заданную палитру цветов. Каждая метка класса заменяется на соответствующий RGB-цвет, и результат возвращается как изображение.

Функция **convert_from_color** преобразует RGB-изображение (arr_3d) в one-hot encoding меток классов, используя заданную палитру цветов.

```
def convert_from_color(arr_3d, palette=invert_palette):
    palette_colors = np.array(list(palette.keys()))
    palette_labels = np.array(list(palette.values()))
```

palette_colors – массив RGB-цветов, извлеченных из ключей словаря **palette**.

palette_labels – массив меток классов, извлеченных из значений словаря palette.

Массив **distances** имеет форму (height, width, num_colors), где каждый элемент – это расстояние между пикселем и цветом палитры.

arr_3d[:, :, np.newaxis, :] добавляет новую ось к arr_3d, чтобы можно было вычислить разницу между каждым пикселем и каждым цветом палитры.

arr_3d[:, :, np.newaxis, :] – palette_colors вычисляет разницу между каждым пикселем и каждым цветом палитры.

np.linalg.norm(..., axis=3) вычисляет евклидово расстояние между каждым пикселем и каждым цветом палитры по оси RGB (ось 3).

```
    distances = np.linalg.norm(arr_3d[:, :, np.newaxis, :] - palette_colors,
axis=3)
```

np.argmin(distances, axis=2) находит индекс ближайшего цвета для каждого пикселя (по оси 2, которая соответствует цветам палитры).

```
    nearest_color_indices = np.argmin(distances, axis=2)
```

palette_labels[nearest_color_indices] использует индексы ближайших цветов для получения соответствующих меток классов.

```
    arr_2d = palette_labels[nearest_color_indices]
```

np.eye(num_classes)[arr_2d] использует массив **arr_2d** для индексации строк единичной матрицы размера (num_classes, num_classes), создавая one-hot encoding.

```
    num_classes = len(palette)
    one_hot = np.eye(num_classes)[arr_2d]
    return one_hot
```

Так как в обучающей выборке изображения могут быть разного размера мы их приводим к одному размеру, обрезаем до 500x500 пикселей.

```
for i in range(len(X_train)):
    X_train[i] = np.array(X_train[i])[:480, :480]
    Y_train[i] = np.array(Y_train[i])[:480, :480]

print()
for j in range(len(X_test)):
    X_test[j] = np.array(X_test[j])[:480, :480]
    Y_test[j] = np.array(Y_test[j])[:480, :480]
```

Следующий блок кода подготавливает данные для использования нейронной сетью.

```
.
.
Допишите код
.
.
```

В результате выполнения данного кода будут записаны данные после предварительной обработки:

X_train_pred – исходные изображения из обучающей выборки данных.

Y_train_pred – сегментированные исходные изображения из обучающей выборки.

X_test_pred – исходные изображения из тестовой выборки данных.

Y_test_pred – сегментированные исходные изображения из тестовой выборки.

4. Реализация топологии нейронной сети U-Net

На рисунке 3 представлена структура U-Net. Как видно по рисунку, сеть содержит сжимающий путь (слева) и расширяющий путь (справа). На вход сети поступает цветное изображение. Далее на каждом слое увеличивается количество каналов признаков. Сжимающий путь похож на типичную свёрточную сеть, он содержит два подряд свёрточных слоя, после которых идет слой ReLU и Pooling размером 2×2 с шагом 2. Следом за ним так же идут два подряд свёрточных слоя с ReLU и Pooling. Далее идут три подряд сверточных слоя с ReLU и Pooling. Это повторяется трижды.

После сжимающего пути следует расширяющий путь. Каждый шаг расширяющего пути содержит слой, обратный операции Pooling, который расширяет карту признаков, после которого следует свертка 2x2, которая уменьшает количество каналов признаков. После идет конкатенация с соответствующим образом из сжимающего пути и две свертки 3x3, после каждой из которых идет ReLU. На последнем слое используется свертка 1x1 для приведения каждого 64-компонентного вектора признаков до требуемого количества классов.

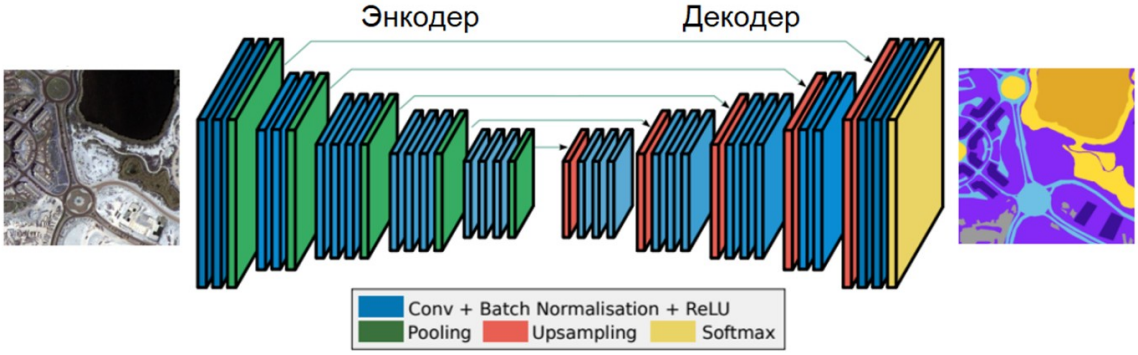


Рисунок 3. Пример модели нейронной сети U-Net

Для начала импортируем необходимые модули из библиотек.

```
from keras.models import *
from keras.layers import *
from keras.optimizers import Adam
from keras.metrics import MeanIoU
```

Для реализации сети U-Net будем использовать функциональный подход. Оригинальная сеть U-Net содержит 23 сверточных слоя плюс в каждом из них применяется пакетная нормализация и функция активации ReLU, 4 слоя Max pooling, 4 слоя Upsampling, в конце функция активации Softmax. В сумме получается 31 слой. В данной работе мы попробуем реализовать упрощенный вариант сети U-Net, которая будет состоять из 15 слоев: 11 сверточных слоев, в каждом из которых применяется функция активации ReLU; 2 слоя Max pooling; 2 слоя Upsampling. Дважды выполняется операция конкатенации данных. Так же в данной сети не применяется пакетная нормализация данных.

```
.
.
Допишите код
.
.
```

Conv2D – создание сверточного слоя со следующими параметрами: количество плоскостей (фильтров), размер ядра свертки, **activation** – функция активации, **padding** – параметр "same" означает не изменять размер изображения после его обработки сверточным слоем.

MaxPooling2D – создание слоя Max-pooling. Окном 2x2 сканируется предыдущий слой, выбирает в каждой такой области максимальный элемент и записывает его в текущий слой. Уменьшает размер входного слоя в 2 раза.

UpSampling2D – операция обратная Max-pooling. Слой увеличивается в 2 раза относительно предыдущего слоя.

Concatenate – конкатенация выходов двух слоев.

ReLU – функция активации, выполняющая простое нелинейное преобразование поданных на вход данных (x). Возвращает x, если x > 0 и 0 в противном случае. Отличается высокой скоростью вычисления.

Softmax – функция активации, выполняющая нелинейное преобразование на основе математической функции.

В задачах сегментации, помимо accuracy, полезно использовать другие метрики, такие как **IoU (Intersection over Union)**, которая лучше отражает качество сегментации.

5. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод .fit(), используя блок кода ниже:

```
.
.
Допишите код
.
.
```

6. Тестирование нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

7. Результат работы нейронной сети

Чтобы сегментировать изображения, запустите следующий код:

```
.
.
Допишите код
.
.
```

Выведем несколько изображений из тестовой выборки и результат их сегментации.

```
num_images = 8
fig, axes = plt.subplots(3, num_images, figsize=(15, 7))
for i in range(num_images):
    axes[0, i].imshow(np.array(X_test[i]))
    axes[0, i].set_title(f"Изображение {i}")
    axes[0, i].axis('off')

for i in range(num_images):
    axes[1, i].imshow(np.array(Y_test[i]))
    axes[1, i].set_title(f"Маска {i}")
    axes[1, i].axis('off')
```

```
for i in range(num_images):
    predicted_labels = np.argmax(out[i], axis=-1)
    axes[2, i].imshow(convert_to_color(predicted_labels))
    axes[2, i].set_title(f"Результат {i}")
    axes[2, i].axis('off')

plt.tight_layout()
plt.show()
```

На рисунке 4 представлены результаты сегментации исходного изображения. Верхний ряд – исходные изображения, средний ряд – сегментированные маски, нижний ряд – полученный результат сегментации.

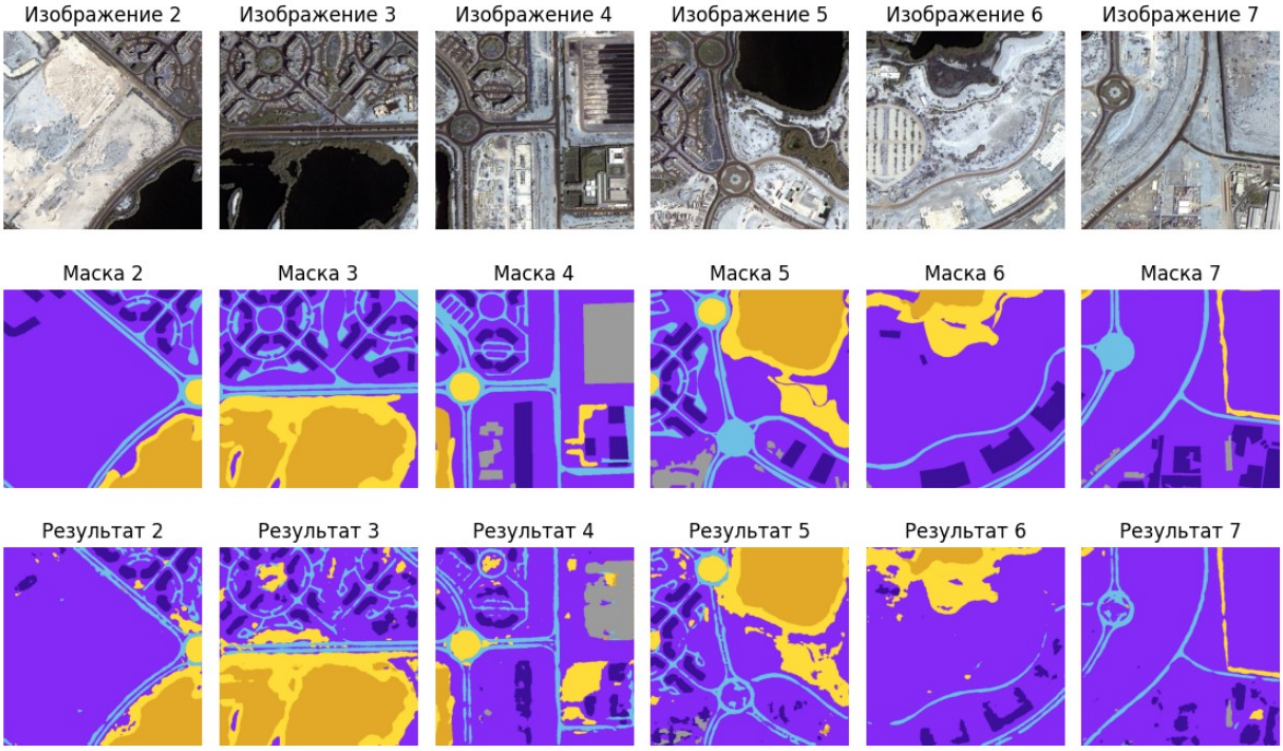


Рисунок 4. Полученный результат сегментации

Вывод

В результате выполнения данной лабораторной работы Вы получили навыки сегментации снимков дистанционного зондирования земли на основе модели нейронной сети U-Net.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания – 0 баллов.

Большей частью выполнено – 2 балл.

Выполнено полностью – 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету – 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований – 2 балл.

Соответствует всем требованиям – 4 балла.